

1 Especificación y procesamiento del lenguaje

Un compilador es un programa que recibe como entrada un programa escrito en un **lenguaje fuente** y produce como salida un programa equivalente escrito en un **lenguaje destino**. Durante este proceso, el compilador analiza la estructura del programa fuente, determina su significado y construye una representación adecuada para generar el programa destino.

De acuerdo con la organización clásica presentada en el capítulo 1 del *Dragon Book*, el proceso de compilación puede dividirse en varias fases:

1. análisis léxico;
2. análisis sintáctico;
3. análisis semántico;
4. generación de código intermedio;
5. optimización de código;
6. generación de código destino.

Estas fases no deben entenderse necesariamente como programas independientes. Una implementación concreta puede agrupar varias de ellas en una misma pasada sobre el programa.

De forma general, el proceso puede representarse como:

Programa fuente → Análisis → Representación intermedia → Optimización → Código destino

En nuestro caso, el lenguaje destino será C, por lo que el proceso completo terminará produciendo un programa escrito en C que posteriormente podrá ser compilado por un compilador de C.

Como ejemplo se utilizará el siguiente programa:

```
int x = 10 + 20;
```

1.1 Análisis léxico

El análisis léxico constituye la primera fase del compilador. Su función es leer los caracteres del programa fuente y agruparlos en unidades denominadas **lexemas**. Cada lexema es una secuencia de caracteres que pertenece a alguna categoría léxica del lenguaje.

El analizador léxico reconoce cada lexema y produce un **token** que representa su categoría. De esta manera, el análisis léxico establece una separación entre la representación concreta escrita por el programador y la categoría que interesa a las fases posteriores.

Por ejemplo, para:

```
int x = 10 + 20;
```

podemos identificar los siguientes lexemas:

```
int
x
=
10
+
20
;
```

Cada uno de estos lexemas pertenece a una categoría léxica:

```
INT
IDENTIFIER
ASSIGN
INTEGER
PLUS
INTEGER
SEMICOLON
```

Por lo tanto, el análisis léxico responde principalmente a la pregunta:

¿Qué unidades léxicas aparecen en la secuencia de caracteres del programa fuente?

El analizador léxico no determina si esas unidades pueden combinarse correctamente. Esa responsabilidad corresponde al analizador sintáctico.

1.2 Lexemas, tokens y patrones

Es importante distinguir tres conceptos relacionados.

Un **lexema** es una secuencia concreta de caracteres que aparece en el programa fuente. Por ejemplo, en:

```
int contador = 25;
```

los lexemas incluyen:

```
int
contador
=
25
;
```

Un **token** es la categoría léxica asignada a un lexema. Por ejemplo:

```
int      -> INT
contador -> IDENTIFIER
=        -> ASSIGN
25       -> INTEGER
;        -> SEMICOLON
```

Finalmente, un **patrón** es una regla que describe qué secuencias de caracteres pueden formar lexemas pertenecientes a un determinado token.

Para nuestro lenguaje se puede definir inicialmente:

Token	Patrón o lexema	Ejemplo
INT	int	int
IDENTIFIER	[a-zA-Z_] [a-zA-Z0-9_]*	x, contador
INTEGER	[0-9]+	10, 250
ASSIGN	=	=
PLUS	+	+
SEMICOLON	;	;

Table 1: Especificación léxica inicial del lenguaje

Por lo tanto, para el programa:

```
int x = 10 + 20;
```

el analizador léxico podría producir una secuencia como:

```
INT
IDENTIFIER(x)
ASSIGN
INTEGER(10)
PLUS
INTEGER(20)
SEMICOLON
```

Los tokens que poseen un valor asociado, como los identificadores y los literales enteros, pueden transportar información adicional. Por ejemplo:

```
IDENTIFIER(x)
INTEGER(10)
```

El identificador también puede representarse mediante una referencia a una entrada de la tabla de símbolos:

```
(id, 1)
```

donde *id* representa la categoría del token y 1 identifica la entrada correspondiente en la tabla de símbolos.

1.3 Análisis sintáctico

El análisis sintáctico recibe la secuencia de tokens producida por el analizador léxico y determina si dicha secuencia puede derivarse de acuerdo con la gramática del lenguaje.

Mientras que el análisis léxico determina qué unidades aparecen en el programa, el análisis sintáctico determina cómo pueden combinarse esas unidades.

Por ejemplo, la secuencia:

```
INT IDENTIFIER ASSIGN INTEGER PLUS INTEGER SEMICOLON
```

puede corresponder a una declaración válida:

```
int x = 10 + 20;
```

En cambio:

```
INT IDENTIFIER ASSIGN PLUS INTEGER SEMICOLON
```

contiene tokens válidos individualmente, pero su organización no corresponde a una construcción permitida por la gramática.

Por lo tanto, el análisis sintáctico responde a la pregunta:

¿Cómo se combinan los tokens para formar construcciones válidas del lenguaje?

1.4 Especificación sintáctica

La sintaxis del lenguaje se especifica mediante una gramática libre de contexto. Para el lenguaje propuesto podemos definir inicialmente:

$$\begin{aligned} \langle \text{programa} \rangle &\rightarrow \langle \text{declaracion} \rangle \\ \langle \text{declaracion} \rangle &\rightarrow \langle \text{tipo} \rangle \text{ IDENTIFIER ASSIGN } \langle \text{expresion} \rangle \text{ SEMICOLON} \\ \langle \text{tipo} \rangle &\rightarrow \text{INT} \\ \langle \text{expresion} \rangle &\rightarrow \langle \text{expresion} \rangle \text{ PLUS } \langle \text{termino} \rangle \\ \langle \text{expresion} \rangle &\rightarrow \langle \text{termino} \rangle \\ \langle \text{termino} \rangle &\rightarrow \text{INTEGER} \end{aligned}$$

Esta gramática establece qué secuencias de tokens forman declaraciones y expresiones válidas.
Para:

```
int x = 10 + 20;
```

el analizador sintáctico puede construir una estructura equivalente a:

```
declaracion
|-- tipo -> INT
|-- IDENTIFIER -> x
|-- ASSIGN -> =
|-- expresion
|   |-- expresion
|   |   '-- termino
|   |       '-- INTEGER -> 10
|   |-- PLUS
|   '-- termino
|       '-- INTEGER -> 20
'-- SEMICOLON
```

Esta estructura constituye una representación de la organización sintáctica del programa.

1.5 Árbol de sintaxis abstracta

A partir del análisis sintáctico puede construirse un **árbol de sintaxis abstracta** (AST). A diferencia del árbol de derivación completo, el AST elimina detalles sintácticos que no son necesarios para las fases posteriores y conserva principalmente la estructura semánticamente relevante.

Para:

```
int x = 10 + 20;
```

un AST puede representarse como:

```
Declaration
|-- type: Int
|-- name: x
'-- value: BinaryExpression
    |-- left: IntegerLiteral(10)
    |-- operator: +
    '-- right: IntegerLiteral(20)
```

El AST permite que las siguientes fases trabajen sobre una representación estructurada del programa en lugar de procesar directamente los caracteres del código fuente.

Además, la estructura del árbol permite representar propiedades sintácticas como la precedencia de operadores. Por ejemplo:

```
10 + 20 * 3
```

debe interpretarse como:

$$10 + (20 \times 3)$$

y no como:

$$(10 + 20) \times 3$$

El AST representa esta diferencia mediante la posición de los nodos dentro del árbol.

1.6 Análisis semántico

El hecho de que un programa sea sintácticamente correcto no implica que sea válido desde el punto de vista semántico.

El análisis semántico utiliza el árbol sintáctico y la información asociada a los identificadores para comprobar restricciones que no pueden expresarse adecuadamente mediante la gramática.

Por ejemplo:

```
int x = "hola";
```

puede tener una estructura sintáctica válida, ya que presenta la forma general:

```
tipo + identificador + asignacion + expresion
```

Sin embargo, la asignación puede ser semánticamente incorrecta porque una expresión de tipo cadena no puede asignarse directamente a una variable de tipo `int`.

Para una declaración:

```
int x = e;
```

podemos establecer inicialmente las siguientes reglas semánticas:

1. x debe ser un identificador válido.
2. x no debe haber sido declarado previamente en el mismo ámbito.
3. La expresión e debe producir un valor de tipo `int`.
4. El valor producido por e se asigna a x .
5. La variable x queda registrada como una variable de tipo `int`.

Para la expresión:

```
10 + 20
```

se puede determinar:

`10 : int`

`20 : int`

`10 + 20 : int`

Por lo tanto:

```
int x = 10 + 20;
```

es semánticamente válida.

1.7 Tabla de símbolos

La **tabla de símbolos** es una estructura de datos utilizada por distintas fases del compilador para almacenar información sobre los nombres que aparecen en el programa.

No constituye necesariamente una fase independiente del compilador, sino un servicio utilizado principalmente durante el análisis semántico y, posteriormente, durante la generación de código.

Entre los atributos que puede contener una entrada se encuentran:

- nombre;
- tipo;
- ámbito;
- ubicación;
- información adicional necesaria para la generación de código.

Para:

```
int x = 10 + 20;
```

podemos obtener:

Nombre	Tipo	Ámbito
x	int	local

Table 2: Ejemplo de tabla de símbolos

Cuando el analizador léxico encuentra el lexema **x**, puede producir un token asociado a una entrada de esta tabla:

```
IDENTIFIER -> (id, 1)
```

Las fases posteriores pueden utilizar esta referencia para obtener información sobre el identificador.

1.8 Manejo de errores

El compilador también debe detectar y comunicar errores encontrados durante el procesamiento del programa.

Los errores pueden aparecer en distintas fases. Por ejemplo:

- un carácter que no pertenece al lenguaje puede producir un error léxico;
- una secuencia de tokens que no puede derivarse mediante la gramática puede producir un error sintáctico;
- una operación incompatible con los tipos puede producir un error semántico.

El manejo de errores es una actividad transversal al proceso de compilación. El objetivo no es solamente detectar un error, sino también proporcionar información suficiente para identificar su ubicación y naturaleza.

1.9 Generación de código intermedio

Después del análisis, el compilador puede construir una **representación intermedia** (IR).

La representación intermedia proporciona una forma de separar el lenguaje fuente del lenguaje o arquitectura destino. En lugar de generar directamente código específico de una máquina, el compilador representa las operaciones del programa mediante una forma intermedia.

Para:

```
int x = 10 + 20;
```

una representación intermedia sencilla podría ser:

```
t1 = 10 + 20
x = t1
```

La IR representa las operaciones necesarias sin depender todavía de una arquitectura específica.

Una ventaja importante de esta separación es que permite modificar independientemente el front-end y el back-end del compilador.

1.10 Optimización de código

Una vez construida la representación intermedia, pueden aplicarse transformaciones que preserven el significado del programa pero mejoren sus propiedades.

Por ejemplo:

```
t1 = 10 + 20
x = t1
```

puede transformarse en:

```
x = 30
```

Esta transformación corresponde a la evaluación de una expresión constante, conocida como *constant folding*.

Las optimizaciones pueden realizarse sobre diferentes aspectos del programa, como expresiones, instrucciones, flujo de control o utilización de valores.

La optimización no modifica el significado observable del programa. Su objetivo es obtener una representación equivalente que pueda ser ejecutada de manera más eficiente.

1.11 Generación de código destino

La última fase principal del compilador transforma la representación intermedia en el lenguaje destino.

En un compilador tradicional, el lenguaje destino puede ser directamente un lenguaje ensamblador o código máquina de una arquitectura determinada.

Conceptualmente, esta fase debe resolver cuestiones como:

- qué instrucciones representan cada operación;
- dónde almacenar los valores;
- qué registros utilizar;
- cómo representar las operaciones de control;
- cómo organizar la memoria necesaria.

El proceso puede representarse como:

IR → selección de instrucciones → asignación de registros → código destino

1.12 Front-end y back-end

Las fases del compilador pueden agruparse conceptualmente en **front-end** y **back-end**.

El front-end depende principalmente del lenguaje fuente. Sus responsabilidades incluyen:

- análisis léxico;
- análisis sintáctico;
- análisis semántico;
- generación de una representación intermedia.

El back-end recibe la representación intermedia y se ocupa de transformarla en código del lenguaje destino, incluyendo las optimizaciones y decisiones relacionadas con la generación de código.

De forma simplificada:

$$\text{Fuente} \rightarrow \boxed{\text{Front-end}} \rightarrow \text{IR} \rightarrow \boxed{\text{Back-end}} \rightarrow \text{Destino}$$

Esta separación permite que diferentes lenguajes fuente compartan un mismo back-end:

$$\begin{array}{lcl} \text{Lenguaje A} & \rightarrow & \text{Front-end A} \\ \text{Lenguaje B} & \rightarrow & \text{Front-end B} \end{array} \rightarrow \text{IR} \rightarrow \text{Back-end}$$

De forma similar, un mismo front-end podría utilizarse con distintos back-ends:

$$\text{Fuente} \rightarrow \text{Front-end} \rightarrow \text{IR} \rightarrow \begin{cases} \text{Back-end x86} \\ \text{Back-end ARM} \end{cases}$$

Esta separación constituye una de las principales ventajas de utilizar una representación intermedia.

1.13 Fases y pasadas

Las fases representan las distintas tareas lógicas que debe realizar el compilador. Sin embargo, estas fases no determinan necesariamente cómo se organiza la implementación.

Una implementación puede realizar varias fases durante una misma **pasada** sobre el programa. Por ejemplo, una implementación puede realizar:

$$\text{Fuente} \rightarrow \text{análisis léxico} \rightarrow \text{análisis sintáctico} \rightarrow \text{análisis semántico} \rightarrow \text{IR}$$

como una primera pasada.

Posteriormente puede realizar:

$$\text{IR} \rightarrow \text{IR optimizada}$$

y finalmente:

$$\text{IR optimizada} \rightarrow \text{código destino}$$

Por lo tanto, las **fases** describen las tareas conceptuales del compilador, mientras que las **pasadas** describen cómo se organizan esas tareas durante su implementación.

1.14 Herramientas para la construcción de compiladores

Las diferentes fases de un compilador pueden implementarse manualmente o mediante herramientas especializadas.

Entre las herramientas utilizadas habitualmente se encuentran:

- **generadores de analizadores léxicos**, que producen analizadores a partir de especificaciones de tokens y patrones;
- **generadores de analizadores sintácticos**, que producen analizadores a partir de una gramática;
- **motores de traducción orientados a la sintaxis**, que permiten asociar acciones o reglas de traducción a estructuras sintácticas;
- **generadores de código**, que facilitan la construcción de back-ends;
- **herramientas de análisis de flujo**, utilizadas para obtener información necesaria para distintas optimizaciones;
- **kits de construcción de compiladores**, que proporcionan componentes para implementar varias de estas tareas.

Estas herramientas no constituyen fases adicionales. Son mecanismos que facilitan la implementación de las fases existentes.

1.15 Proceso completo

El proceso completo puede resumirse de la siguiente manera:

Fuente $\rightarrow$ Análisis léxico $\rightarrow$ Tokens

Tokens $\rightarrow$ Análisis sintáctico $\rightarrow$ AST

AST $\rightarrow$ Análisis semántico $\rightarrow$ AST validado

AST validado $\rightarrow$ Código intermedio $\rightarrow$ Optimización

IR optimizada $\rightarrow$ Generación de código $\rightarrow$ Código destino

En cada una de estas fases intervienen estructuras auxiliares, como la tabla de símbolos y los mecanismos de manejo de errores.

2 Uso de C como lenguaje destino

En nuestro compilador se utilizará C como lenguaje destino. Esto significa que nuestro compilador no generará directamente código máquina ni ensamblador. En cambio, generará un programa escrito en C que represente las operaciones del lenguaje fuente.

El proceso será:

Lenguaje fuente $\rightarrow$ Análisis léxico $\rightarrow$ Análisis sintáctico $\rightarrow$ Análisis semántico $\rightarrow$ IR $\rightarrow$ Código C $\rightarrow$ Compilador

Por lo tanto, el objetivo de nuestro back-end será traducir la representación intermedia a código C válido.

Por ejemplo, si nuestra representación intermedia contiene:

```
t1 = 10 + 20
x = t1
```

el back-end podría generar:

```
int x;
x = 10 + 20;
```

El compilador de C se encargará posteriormente de realizar las etapas necesarias para producir código específico de la plataforma.

Esta decisión simplifica considerablemente la construcción del back-end. Los compiladores de C existentes proporcionan optimizaciones maduras y soporte para múltiples arquitecturas y plataformas. Por lo tanto, nuestro proyecto no necesita implementar desde cero las instrucciones, registros, convenciones y características específicas de cada arquitectura.

Si, en cambio, generáramos código máquina directamente, deberíamos implementar un back-end específico para cada arquitectura que quisiéramos soportar.

La utilización de C como lenguaje destino permite entonces concentrar el trabajo del proyecto en el diseño y procesamiento de nuestro propio lenguaje, delegando al compilador de C la generación final de código para la plataforma correspondiente.

3 Conclusión

A partir de este modelo se puede definir el desarrollo de nuestro compilador como una secuencia de etapas claramente diferenciadas.

En primer lugar, se debe definir la especificación léxica del lenguaje, determinando sus tokens, lexemas y patrones. Luego se debe establecer la gramática que determina cómo pueden combinarse esos tokens. Sobre esta estructura se incorporarán las reglas semánticas y la tabla de símbolos necesarias para validar los programas.

Una vez definido el front-end, se establecerá una representación intermedia que permita expresar las operaciones del lenguaje de manera independiente del código C generado. Finalmente, se implementará el back-end encargado de traducir dicha representación a C.

De esta manera, el desarrollo seguirá la organización clásica de un compilador:

$$\text{Lenguaje} \rightarrow \text{Léxico} \rightarrow \text{Sintaxis} \rightarrow \text{Semántica} \rightarrow \text{IR} \rightarrow \text{C}$$

Esta estructura permite construir el compilador progresivamente y mantener separadas las responsabilidades de cada etapa.